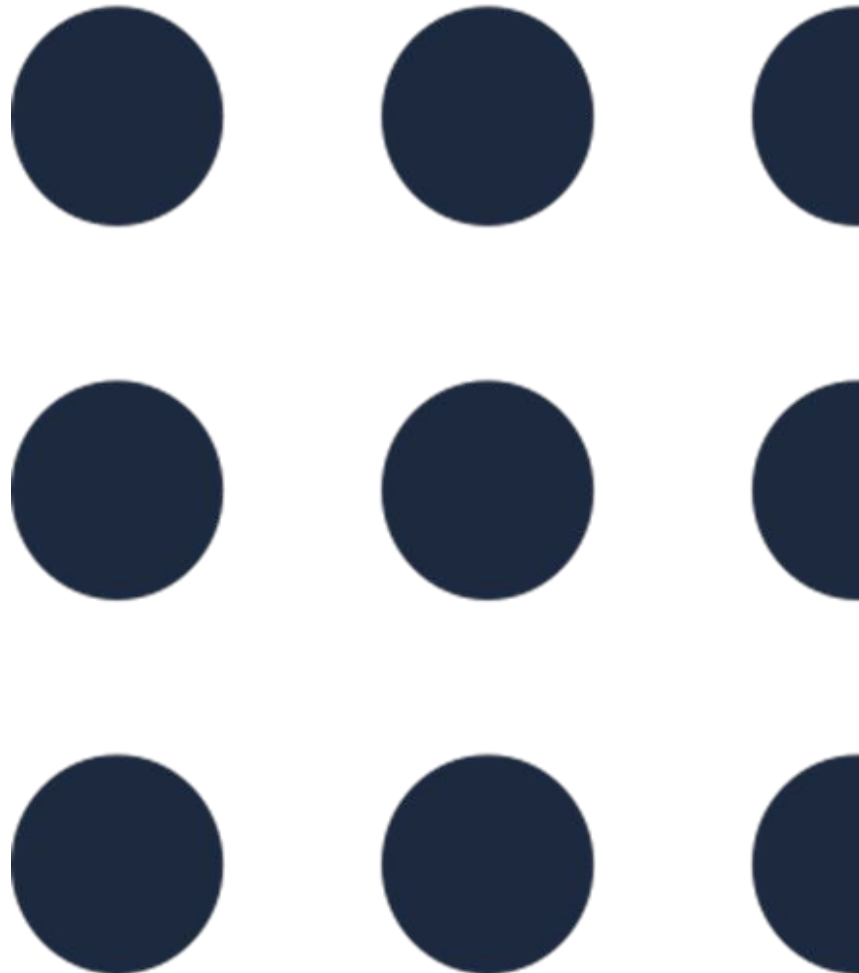


RRM

Cvičenie 7

Marek Čornák
marek.cornak@stuba.sk

Jakub Ivan
jakub.ivan@stuba.sk



Obsah

1. Inverzná kinematika?
2. ROS parameter
3. URDF parser

Parametre v ROS

- nastavenia program (node), ktoré určujú jeho správanie bez potreby meniť zdrojový kód
 - názov frame-u
 - maximálna rýchlosť
 - boolean prepínač `use_sim_time`
 - PID konštanty
 - cesta k súboru alebo modelu
 - text URDF v parametri `robot_description`
- V ROS 2 si **každý node spravuje svoje vlastné parametre**. To znamená, že parameter nepatrí celému systému, ale vždy konkrétnemu uzlu. ROS 2 podporuje typy ako integer, double, bool, string a aj polia týchto typov.

Parametre v ROS – využitie a výhody

- Parametre používame najmä na:
 - konfiguráciu node-u pri štarte
 - jednoduché prepínanie režimov
 - ladenie správania aplikácie
 - zmenu nastavení bez rekompilácie
 - načítanie konfigurácie zo súboru YAML
- **ten istý node môžeme spustiť v rôznych režimoch len zmenou parametrov**
- **IBA NA KONFIGURÁCIU** – nie na prenos dát

štúdium: [Parameters — ROS 2 Documentation: Jazzy documentation](#)

Parametre v ROS2 - lifecycle

Typický postup práce s parametrami v C++ node-e:

1. parameter deklarujeme
2. načítame jeho hodnotu
3. používame ho v logike node-u
4. prípadne dovoľíme jeho zmenu za behu
5. hodnoty môžeme načítať z CLI alebo zo YAML súboru

Parameter môžeme nastaviť:

- priamo v kóde ako default
- pri spustení node-u cez `--ros-args -p`
- cez YAML súbor
- za behu cez `ros2 param set`
- ROS 2 podporuje všetky tieto spôsoby práce s parametrami.

Parametre – CLI nástroje

ROS 2 má nástroj `ros2 param`, ktorý vie parametre:

- Vypísať, získať, zmeniť, opísať, uložiť, načítať

CLI príkazy:

```
ros2 param list
ros2 param list /my_node
ros2 param get /my_node speed
ros2 param set /my_node speed 2.0
ros2 param describe /my_node speed
ros2 param dump /my_node
ros2 param load /my_node params.yaml
```

[Using the `ros2 param` command-line tool — ROS 2 Documentation: Jazzy documentation](#)

Parametre - spustenie procesu

Parametre môžeme poslať už pri spustení node-u.

```
ros2 run my_package my_node --ros-args -p speed:=1.5 -p enabled:=true
```

alebo zo súboru:

```
ros2 run my_package my_node --ros-args --params-file params.yaml
```

V ROS 2 musia byť ROS-špecifické argumenty uvedené za `--ros-args`. To platí aj pre parametre.

Príklad v C++

```
#include <rclcpp/rclcpp.hpp>
#include <string>

class DemoNode : public rclcpp::Node
{
public:
    DemoNode() : Node("demo_node")
    {
        this->declare_parameter<std::string>("robot_name", "UR5e");
        this->declare_parameter<double>("speed", 1.0);
        this->declare_parameter<bool>("enabled", true);

        robot_name_ = this->get_parameter("robot_name").as_string();
        speed_ = this->get_parameter("speed").as_double();
        enabled_ = this->get_parameter("enabled").as_bool();

        RCLCPP_INFO(this->get_logger(),
            "robot_name=%s, speed=%.2f, enabled=%s",
            robot_name_.c_str(),
            speed_,
            enabled_ ? "true" : "false");
    }

private:
    std::string robot_name_;
    double speed_;
    bool enabled_;
};

int main(int argc, char ** argv)
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<DemoNode>());
    rclcpp::shutdown();
    return 0;
}
```

`declare_parameter(...)` vytvorí parameter a nastaví default hodnotu

`get_parameter(...)` načíta aktuálnu hodnotu parametra z tohto node-u

tento spôsob funguje len pre vlastné parametre daného uzla (node)!!!

Súbor YAML

- Parametre sa veľmi často ukladajú do YAML súboru.
- Príklad:

```
demo_node:  
  ros__parameters:  
    robot_name: "UR5e"  
    speed: 1.2  
    enabled: true
```

- Spustenie:

```
ros2 run my_package demo_node --ros-args --params-file params.yaml
```

[Passing ROS arguments to nodes via the command-line — ROS 2 Documentation: Jazzy documentation](#)

C++ parameter callback

- Ak chceme parameter validovať alebo reagovať okamžite na zmenu, použijeme callback.

```
#include <rcl_interfaces/msg/set_parameters_result.hpp>

callback_handle_ = this->add_on_set_parameters_callback(
  [this](const std::vector<rclcpp::Parameter> & params)
  {
    rcl_interfaces::msg::SetParametersResult result;
    result.successful = true;
    result.reason = "success";

    for (const auto & param : params) {
      if (param.get_name() == "speed") {
        double value = param.as_double();
        if (value < 0.0 || value > 5.0) {
          result.successful = false;
          result.reason = "speed must be in range <0.0, 5.0>";
          return result;
        }
      }
    }
    return result;
  });
```

- callback sa zavolá pri pokuse o zmenu parametra
- vieme skontrolovať, či nová hodnota dáva zmysel
- vieme zmenu povoliť alebo zamietnuť

Parametre - priestor

- `get_parameter()` vie vyčítať parametre iba z vlastného procesu/node
`this->get_parameter("robot_description")` □ parameter deklarovaný v node, ak by sme chceli vyčítať parameter inej node, toto nestačí
- V ROS 2 parametre **nie sú globálne spoločné**. Každý node má svoje vlastné parametre!

Načítanie parametrov z inej node

```
#include <rclcpp/rclcpp.hpp>
#include <rclcpp/parameter_client.hpp>
#include <string>
#include <memory>
#include <chrono>

using namespace std::chrono_literals;

class ReaderNode : public rclcpp::Node
{
public:
    ReaderNode() : Node("reader_node")
    {
        client_ = std::make_shared<rclcpp::SyncParametersClient>(
            this, "/robot_state_publisher");

        if (!client_->wait_for_service(2s)) {
            RCLCPP_ERROR(this->get_logger(),
                "Parameter service /robot_state_publisher nie je dostupna");
            return;
        }

        auto params = client_->get_parameters({"robot_description"});
        if (!params.empty()) {
            std::string urdf = params[0].as_string();
            RCLCPP_INFO(this->get_logger(),
                "Nacitany robot_description, dlzka = %zu", urdf.size());
        }
    }

private:
    std::shared_ptr<rclcpp::SyncParametersClient> client_;
};
```

- rclcpp::SyncParametersClient

- rclcpp::AsyncParametersClient

URDF model parser

- Balík urdf
- Parsovanie premennej string na model URDF

```
#include <urdf/model.h>
#include <string>
#include <iostream>

int main()
{
    std::string urdf_xml = R"(
    <robot name="demo_robot">
        <link name="base_link"/>
    </robot>
    )";

    urdf::Model model;
    bool ok = model.initString(urdf_xml);

    if (ok) {
        std::cout << "URDF was loaded successfully\n";
        std::cout << "Robot name: " << model.getName() << std::endl;
    } else {
        std::cout << "Failed to parse URDF\n";
    }

    return 0;
}
```